ALADYN GUI

User's Guide

From a POSCAR to phonons and the thermal conductivity, in a browser: VASP · SC-SNAPS · FOCEX · THERMACOND

Keivan Esfarjani

v0.1 · November 2026

https://github.com/KeivanS/Anharmonic-lattice-dynamics (GUI)

Contents

1 What the GUI does 2

2 Installation and start 2

3 The page 3

4 A calculation, step by step 3

5 Where the runs go: here, bundle, submit 4

5.1 On the cluster, with a bundle 5

5.2 Cluster settings 5

6 The input files 6

7 More projects from one 6

8 The project folder 6

9 When something goes wrong 7

10 Version history 7

1 What the GUI does

The ALADYN GUI (`aladyn-gui`) takes you, in a web browser, through a whole lattice-dynamics calculation: relaxation of the crystal, SCF and Born charges, supercell snapshots (SC-SNAPS), one DFT run per snapshot, the force-constant fit (FOCEX) with its phonon plots, and the lattice thermal conductivity (THERMACOND). You upload a POSCAR and press buttons; a panel on the left always says which step comes next and what to do.

Underneath it is the FOCEX agent (`FOCEX/AGENT/focex-agent.py`, its own manual `focexagent_manual.pdf`): every GUI project is an agent project folder, the GUI writes the agent's instruction file and runs the agent's tasks one at a time. What the GUI adds:

- a choice, for each DFT stage and for THERMACOND, of *where* it runs: on this computer, as a bundle for a SLURM cluster, or submitted directly when the GUI itself runs on the cluster (Section 5);
- editors for the input files, with a comment after every line and the values that are not known yet marked `PLACEHOLDER` (Section 6);
- stops before FOCEX and before THERMACOND, so that you review their input files first;
- the plots and the SC-SNAPS displacement histograms in the page.

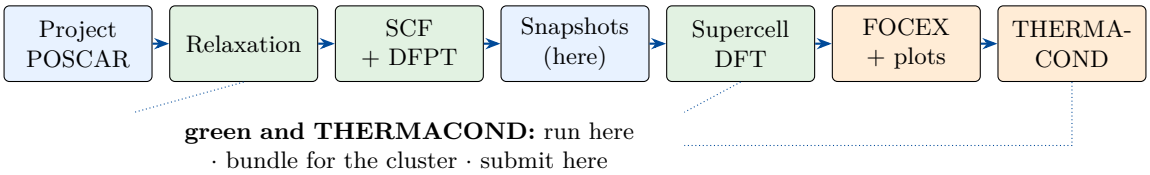


Figure 1: The seven steps of the guide panel. The GUI stops before FOCEX and THERMACOND (orange) until you start them; blue steps are quick and always run where the GUI runs.

2 Installation and start

The GUI is part of the ALADYN repository (`GUI/aladyn-gui.py`). It needs what the FOCEX agent needs (Python with numpy, spglib and pymatgen; VASP-Flow; SC-SNAPS and FOCEX built; a POTCAR library) and the Python package Flask. Set these up once as described in the FOCEX agent manual (`focex-configure.sh`).

Do this: once, after git pull

```
cd ~/PROJECTS/Anharmonic-lattice-dynamics
make python-tools            # links aladyn-gui into ~/ALADYN/bin
pip install flask            # in the agent's Python environment, if it is not
    there
```

On your computer

Do this

```
source ~/myenv/bin/activate              # the agent's Python environment
cd ~/calc && aladyn-gui                  # projects go in this folder
```

Then open http://localhost:5070. Options: `-workdir DIR` (folder of the projects, default the current one), `-port N` (default 5070), `-host` (default 127.0.0.1: only this computer can connect).

On a cluster

Start it on the login node and reach it from your computer through an ssh tunnel. Then the DFT runs can be submitted from the page, and nothing has to be copied back and forth.

Do this

```
ssh -L 5070:localhost:5070 you@cluster      # on your computer; keep this open
cd /scratch/$USER && aladyn-gui             # on the cluster
```

Open http://localhost:5070 on your computer. The button Submit here is active when the GUI finds `sbatch`.

3 The page

The guide (left, always visible) lists the steps in order, each marked *done*, *next*, *waiting* (its inputs are ready and the runs are outstanding), *failed* or *later*; a step done outside (a relaxed POSCAR, results of a cloned project) says *provided*. Below it, **What to do next** names the button to press. It is updated every few seconds while a job runs.

Tab	What it holds
Projects	open a project; a new project from a POSCAR; a new project from the open one
Run	the run buttons, the import of results, the bundles made so far, the cluster settings
Snapshots	the SC-SNAPS summary (supercell, temperature, displacements) and two histograms
Params	the five FOCEX input files
Results	the PDF plots of FOCEX (`05_focex`, also `NOBORN/` and `LASSO/`) and THERMACOND
THERMACOND	the `latdyn.params` of THERMACOND and its run buttons
Setup	the whole instruction file of the project (checked before it is saved)
Log	the output of the running job and `report.log` (one entry per task)

4 A calculation, step by step

1. New project. On *Projects*: a name, the POSCAR (paste it or choose the file), and the main settings:

Field	Meaning (instruction keyword)
Functional	PBE, PBEsol, LDA, R2SCAN or HSE06 (`FUNCTIONAL`)
Accuracy	Accurate: ENCUT 600 eV, EDIFF 10^{-7}; Normal (`ACCURACY`)
DFPT	Born charges: auto (when the SCF shows a gap), on, off (`DFPT`)
DFT code	VASP (others later)
Force-constant ranks	harmonic; harmonic + cubic (needed by THERMACOND); + quartic (`RANKS`)
Shells for ranks 2..8	neighbour shells of each rank (`RANGE`)
Supercell	SMALL, LARGE or 9 integers (`SUPERCELL`)
Snapshots	number of snapshots (`SNAPSHOTS`)
Temperature	of the snapshots; empty: 30 K tuned to an rms displacement of 0.02 Å
THERMACOND job	0 none, 1 linewidths, 2 and 3 thermal conductivity (`THERMACOND_JOB`)
MPI ranks here	VASP and THERMACOND ranks on this computer
More instruction lines	any keyword of the agent or VASP-Flow (`MAGMOM`, `GGA_U`, INCAR blocks)

Tick *This POSCAR is already relaxed* only if it was relaxed with these same DFT settings: the supercell runs must use the settings of the relaxation. Otherwise the GUI relaxes it first. Create project makes the folder, the POTCAR, the relaxation inputs, and the five draft input files.

2. Relaxation. On *Run*: Run here (a small cell relaxes in minutes on a workstation), Make bundle or Submit here. A relaxed POSCAR: Skip: POSCAR is relaxed. Then Continue: the relaxed cell becomes `POSCAR0`, the cell of every later step.

3. SCF and DFPT. The same buttons. With DFPT both go into one bundle or submission; DFPT starts after the SCF and reads its charge density.

4. Snapshots. Continue makes the snapshots (SC-SNAPS) and one VASP folder per snapshot, here; see the *Snapshots* tab. It stops with the supercell runs waiting.

5. Supercell DFT. The expensive step: usually Make bundle (or Submit here on the cluster). All snapshots are submitted at once; a last job writes `FORCEDISP1`, `POSCAR1` and `stress.params`. Import the results, then Continue: the guide moves to FOCEX.

6. FOCEX. On *Params*, read the five files (Section 6), change what you want, then Save and run FOCEX. FOCEX runs here (minutes) and the plots follow (*Results*).

7. THERMACOND. On *THERMACOND*, check line 8 (the job), line 1 (q mesh) and line 6 (temperatures), then Save and run here, Save and make bundle or Save and submit here. The thermal conductivity is in `07_thermacond/kappa_vs_T.dat`; the table of `report.log` (*Log*) gives it at a few temperatures.

5 Where the runs go: here, bundle, submit

Continue does the quick steps here (POSCAR0, snapshots, folders, reading results that came back) and stops at the next DFT stage with its inputs ready (the guide then says *waiting*). It never starts VASP.

Run here runs the next DFT stage on this computer, with the *MPI ranks here*.

Make bundle packs the stage as `bundles/<project>_<task>_<date>.tar.gz` (link on the *Run* tab) for a SLURM cluster.

Submit here submits the same scripts with `sbatch` (the GUI runs on the cluster). When the jobs are done, Continue.

5.1 On the cluster, with a bundle

Do this

```
tar xzf Cu2O_vasp_snaps_20261107-214157.tar.gz     # where the project should live
    (scratch)
cd Cu2O && ./submit_all.sh
squeue -u $USER                                    # until the jobs are gone
./pack_results.sh                                  # -> results_Cu2O_<date>.tar.gz
```

Copy `results_*.tar.gz` back, choose it under *Import results* and press Import results, then Continue. The same folder name is not needed: the top folder of the archive is dropped.

In the bundle	What it is
`submit_all.sh`	submits every waiting folder; DFPT waits for the SCF; the supercells all at once, then the collect job
`<folder>/job.sh`	the SLURM job of that folder (header from the cluster settings)
`<folder>/INCAR, KPOINTS, POSCAR, POTCAR`	the VASP inputs (the POTCAR is built on your computer)
`04_supercells/collect.py`	writes FORCEDISP1, POSCAR1, stress.params and collect.log
`04_supercells/collect_job.sh`	runs collect.py after all supercell jobs (30 min, one task)
`06_dfpt/gap_check.py`	with DFPT auto: the job ends at once when the SCF shows no gap
`pack_results.sh`	packs the files to bring back
`README_HPC.txt`	these commands

`pack_results.sh` packs, for the relaxation, SCF and DFPT, the OUTCAR, CONTCAR, OSZICAR, EIGENVAL, `vasp.out` and the Born-charge files; for the supercells only `FORCEDISP1`, `POSCAR1`, `stress.params`, `collect.log` and each run's OSZICAR and `vasp.out` (`./pack_results.sh -outcars` adds the OUTCARs); for THERMACOND every file of `07_thermacond` but its scratch folder `V3DELTA`.

Note. `collect.py` does what `read_outcar.x` does, in Python with the standard library, so nothing has to be compiled on the cluster. It checks every run (finished, converged), the net force and the positions, and writes nothing if a run is not usable (`collect.log` names it). On the 21 Cu_2O snapshots of 2026-11-07 it gave the same numbers as `read_outcar.x`.

5.2 Cluster settings

On the *Run* tab, kept in the project's `hpc.json`:

Setting	Meaning (default)
Partition, Account	`#SBATCH --partition`, `--account` (standard, none)
Nodes, Tasks per node	per VASP job (1, 40); also the ranks used for KPAR
Walltime (VASP), (THERMACOND)	`#SBATCH --time` (12:00:00, 24:00:00)
Memory	`#SBATCH --mem`, optional
MPI launcher	`srun`, or e.g. `mpirun` (then `-np` nodes × tasks)
VASP / THERMACOND binary	on the cluster (`vasp_std`, `~/ALADYN/bin/tcond-2.0.x`)
python3 on the cluster	for collect.py, gap_check.py, set_efield.py, extract_born.py
Lines before VASP	e.g. `module load vasp/6.4.2`
More #SBATCH lines	anything else your cluster wants

KPAR and NCORE in the INCARs are adjusted to the number of ranks the job runs with (KPAR divides the ranks).

6 The input files

The *Params* tab shows `structure.params`, `latdyn.params`, `kpbs.params`, `dielectric.params` and `default.params` as FOCEX will get them, each line followed by a comment that says what it is. Lines marked `PLACEHOLDER` are filled in by a later step or need your values:

- `structure.params`, before the relaxation: lattice and positions of the unrelaxed POSCAR. When the relaxation is done, they are replaced by those of `POSCAR0`; your other lines (ranks, invariances, shells, . . .) are kept.
- `dielectric.params`, before DFPT: zeros and `born_flag` 0. After DFPT the Born charges are there. You may also type ε_∞ and Z^* yourself (e.g. from a database) and set line 1 to 1.

A file you change is kept in the instruction file and used as it is; Back to the generated file undoes your changes (save afterwards). A saved `dielectric.params` is used instead of the DFPT result.

The *THERMACOND* tab holds the `latdyn.params` of `07_thermacond`: before FOCEX has run it is FOCEX's file with the THERMACOND lines of your settings; afterwards the one FOCEX used. The lines that matter most: 1 the q mesh, 6 the temperatures (Tmin, Tmax, number), 8 the job, 9 the solver of job 2, 11 isotope scattering, 13 the sample length for boundary scattering.

7 More projects from one

New project from the open one (*Projects* tab) keeps the relaxed cell, the SCF and the DFPT (they are marked *provided*) and makes the snapshots again with another temperature, supercell or number of snapshots. A second supercell (`FORCEDISP2` for one FOCEX fit) is a separate project.

8 The project folder

The folder is that of the FOCEX agent (its manual, Section "The project folder": `01_relax`, `02_scf`, `03_snaps`, `04_supercells`, `05_focex`, `06_dfpt`, `07_thermacond`, `report.log`, `state.json`). The GUI adds:

File	What it is
`gui.json`	the MPI ranks here; when the input files were saved
`hpc.json`	the cluster settings
`bundles/`	the bundles made, and the imported results (`imported_*`)
`submit_all.sh`, `pack_results.sh`, `README_HPC.txt`	of the last stage made ready
`<folder>/job.sh`	SLURM job of a waiting folder

9 When something goes wrong

A step is marked failed The guide quotes the message of `report.log`. Fix the cause (the instruction file on *Setup*, or the INCAR of that folder) and press Continue. The agent's `diagnose.py` explains most VASP failures and writes a fix script (its command is in the guide).

The step stays *waiting* The results are not in the folders yet: import them, or (GUI on the cluster) wait until `squeue` is empty, then Continue.

collect.py wrote no FORCEDISP1 `collect.log` names the runs that did not finish or converge. Rerun them on the cluster (`cd 04_supercells/snap_NNN && sbatch job.sh`), then `python3 collect.py` in `04_supercells` and `./pack_results.sh`.

No POTCAR The POTCAR library was not found (`VASP_POTCAR_DIR`); put a POTCAR in the project folder.

Born charges not used The DFPT result failed the agent's checks (charges not summing to zero, ε_∞ outside 1–100, VASP's Zener warning); `report.log` says why. Type the values in `dielectric.params`, or redo DFPT.

10 Version history

0.1 2026-11-08: first version, with the FOCEX agent 1.6 (`VASP_RUN` and `THERMACOND_RUN` external) and `FOCEX/AGENT/bundle.py`. Tested on Cu_2O through all steps with the GUI's own requests (no browser, no cluster yet).